

TECNOLOGÍA SUPERIOR EN DESARROLLO DE SOFTWARE Y PROGRAMACIÓN

Asignatura

Matemáticas Discretas

Nombres y Apellidos

Angel Segundo Morales Males

Docente:

Ing. Richard Jorge Heredia Guagchinga

Fecha de entrega

01/08/2026

1: Cómo se aplica la teoría de conjuntos

Ejemplo de código: En `src/services/validacion.ts`, creamos un conjunto con los nombres de las materias del horario y verificamos si las materias obligatorias forman un subconjunto de este

Registrar Nueva Materia

NOMBRE DE LA MATERIA

Ej. Base de Datos

DÍA

Lunes

INICIOFIN

08:00 a. m.10:00 a. m.

MODALIDAD

Presencial

DIFFICULTAD

Medio

Materias Disponibles (12)

ID	NOMBRE	DÍA	HORA	MODALIDAD	DIFFICULTAD	CREDITOS	PREREQUISITOS	ACCIONES
1	Programación	Viernes	08:00 - 10:00	Virtual	Alta	4	10	Editar Eliminar
2	Programación	Lunes	08:00 - 10:00	Presencial	Medio	4	—	Editar Eliminar
3	Matemáticas	Martes	08:00 - 12:00	Presencial	Medio	5	—	Editar Eliminar
4	Inglés	Miércoles	14:00 - 16:00	Virtual	Baja	2	2	Editar Eliminar
5	Base de datos básica	Martes	10:00 - 11:00	Virtual	Baja	4	—	Editar Eliminar
6	Base de datos avanzada	Viernes	11:00 - 13:00	Virtual	Alta	4	4	Editar Eliminar
7	Matemáticas Discretas	Lunes	10:00 - 12:00	Presencial	Alta	4	2	Editar Eliminar

2. ¿Cómo se aplica álgebra proposicional?

Explicación: Se transformaron las reglas de negocio en proposiciones lógicas que devuelven un valor booleano (true o false), evaluadas mediante operadores de conjunción (&&), disyunción (||) y negación (!).

Ejemplo de código: En la función evaluateSchedule (src/services/validacion.ts), se combinan múltiples condiciones lógicas para determinar si una combinación es válida:

```
76 export function evaluateSchedule(schedule: Course[], configuration: any, allCoursesData: Course[]) {
77   const reasons: string[] = [];
78
79   //
80   if (configuration.numberOfCourses < 1 || configuration.numberOfCourses > 10) {
81     reasons.push("El número de materias debe estar entre 1 y 10.");
82   }
83
84   if (configuration.maximumCredits < 0 || configuration.maximumCredits > 12) {
85     reasons.push("El límite máximo de créditos de estar entre 0 y 12 créditos.");
86   }
87
88   if (configuration.maximumDifficultCourses < 0 || configuration.maximumDifficultCourses > 5) {
89     reasons.push("El máximo de materias difíciles debe estar entre 0 y 5.");
90   }
91
92   const courseSet = getCourseNameSet(schedule);
93   if (!includesRequiredCourses(courseSet, new Set(configuration.requiredCourses || []))) {
94     reasons.push("No contiene todas las materias obligatorias.");
95   }
96
97   // Validar cantidad de materias
98   if (schedule.length !== configuration.numberOfCourses) {
99     reasons.push("La cantidad de materias no coincide con la configuración solicitada.");
100   }
}
```

MATERIAS OBLIGATORIAS

- ☐ Programación (Viernes)
- ☐ Programación (Lunes)
- ☐ Matemáticas (Martes)
- ☐ Inglés (Miércoles)
- ☐ Base de datos básica (Martes)

☒ Evitar cruces ☒ Validar prerrequisitos

Generar y Evaluar Horarios

3. ¿Cómo se aplica la combinatoria?

Explicación: Se utilizó para calcular teóricamente la cantidad de grupos posibles mediante factoriales $C(n,r)$ y para generar físicamente todas las combinaciones posibles de materias sin importar el orden.

Ejemplo de código: En src/services/combinatoria.ts, la función recursiva genera las agrupaciones sin importar que el orden cambie:

```
17 export function generateCombinations<T>(elements: T[], size: number): T[][] {
18     const results: T[][] = [];
19
20     function combine(startIndex: number, currentCombination: T[]) {
21         if (currentCombination.length === size) {
22             results.push([...currentCombination]);
23             return;
24         }
25         for (let index = startIndex; index < elements.length; index++) {
26             currentCombination.push(elements[index]);
27             combine(index + 1, currentCombination);
28             currentCombination.pop();
29         }
30     }
31
32     combine(0, []);
33     return results;
34 }
```

1. Admin. Materias

R

MATERIAS DISPONIBLES (N)
12

MATERIAS POR HORARIO (R)
3

4.Cuál fue la regla lógica principal

Explicación: Es la expresión proposicional compuesta que une todas las validaciones de manera simultánea mediante una gran conjunción ($\wedge$), garantizando que un horario cumpla con todas las restricciones a la vez.

Ejemplo de código: En src/services/validacion.ts, la regla principal se consolida al verificar que ningún arreglo de razones contenga errores y que todas las condiciones sean verdaderas:

```
131     return {  
132         valid: reasons.length === 0,  
133         reasons  
134     };  
135 }
```



Cualquiera

MATERIAS OBLIGATORIAS

- ☐ Programación (Viernes)
- ☐ Programación (Lunes)
- ☐ Matemáticas (Martes)
- ☐ Inglés (Miércoles)
- ☐ Base de datos básica (Martes)

☒ Evitar cruces ☒ Validar prerrequisitos

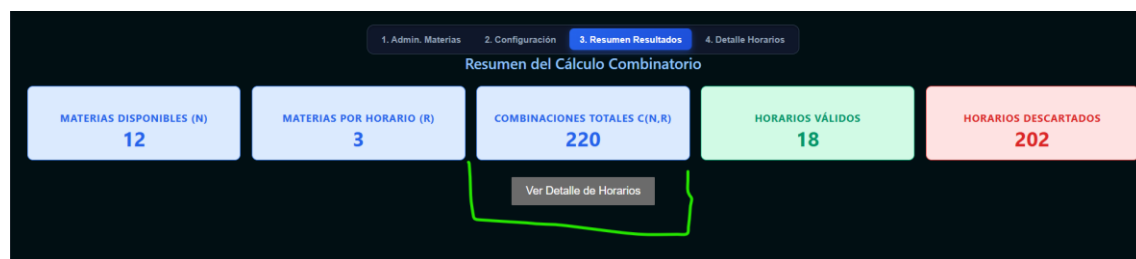
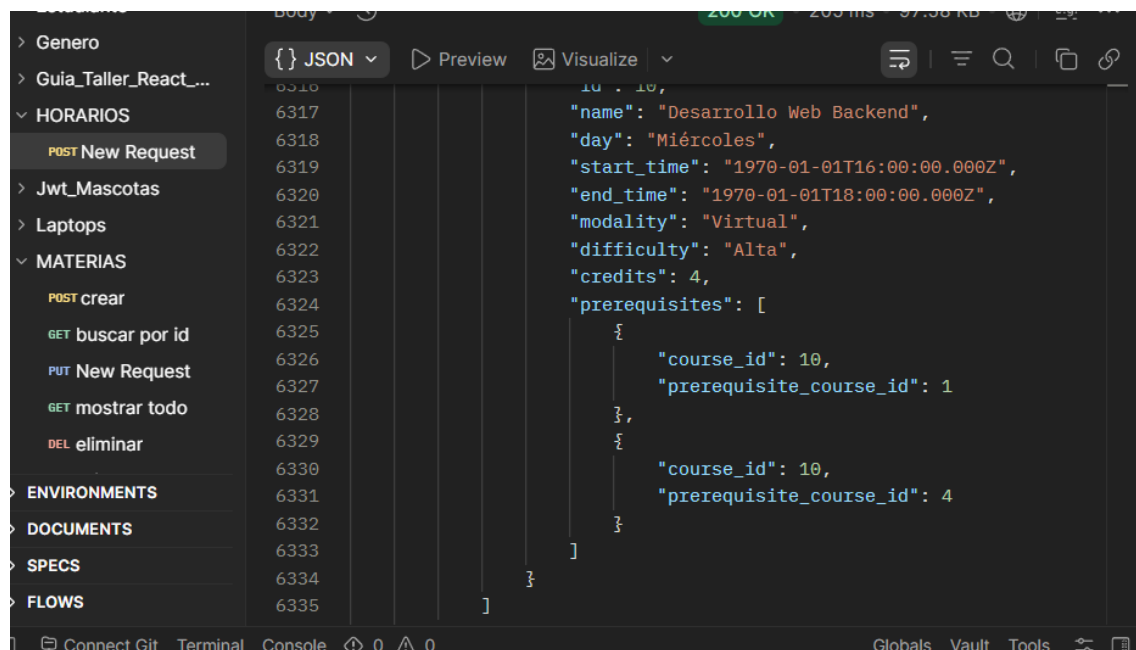
Generar y Evaluar Horarios

5. Cuántas combinaciones generó

Explicación: Es el número total de grupos de tamaño r extraídos del conjunto universal de n materias disponibles en la base de datos.

Ejemplo de código / JSON: Este valor se calcula matemáticamente y se retorna en el controlador del backend (src/services/horarios.ts):

```
18  
19 // 3. Calcular número de combinaciones teóricas  $C(n,r)$  (Paso 4)  
20 const totalCombinations = calculateCombinationCount(courses.length, configuration.numberOfCourses);  
21
```

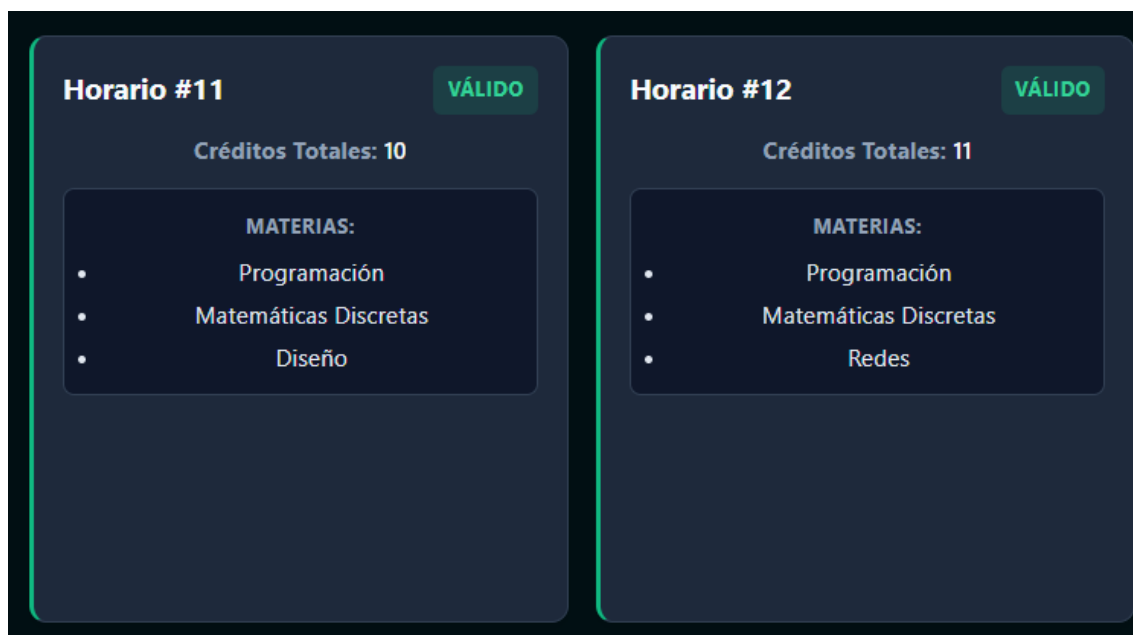
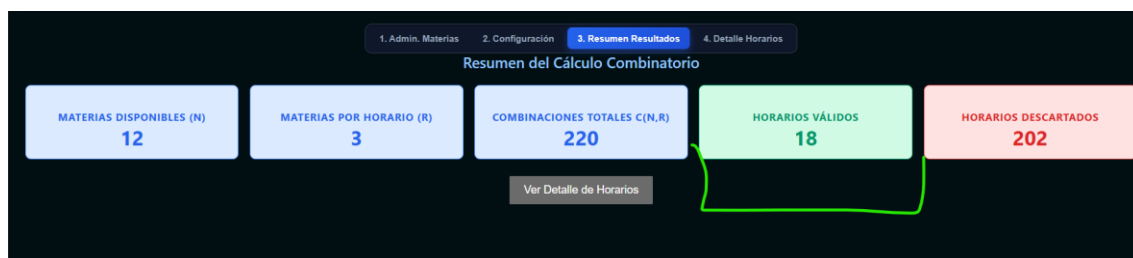


6. Cuántos horarios fueron válidos

Explicación: Corresponde al subconjunto de combinaciones que superaron satisfactoriamente la evaluación lógica sin violar ninguna regla o restricción configurada.

Ejemplo de código: En src/services/horarios.ts, se filtran los horarios cuya evaluación arroja que son válidos:

```
// 6. Separar horarios válidos y descartados (Paso 15 - Sección 37)
const validSchedules = evaluatedSchedules.filter(item => item.evaluation.valid);
```



7. Cuántos horarios fueron descartados

Explicación: Son las combinaciones que incumplieron una o más reglas lógicas, acompañadas de un registro detallado de los motivos de su rechazo.

Ejemplo de código: En src/services/horarios.ts y validacion.ts, se filtran y estructuran los descartes junto con sus razones:

```
33 const discardedSchedules = evaluatedSchedules.filter(item => !item.evaluation.valid);  
34
```



Horario #13

DESCARTADO

Créditos Totales: 12

MATERIAS:

- Programación
- Matemáticas Discretas
- Desarrollo Web Backend

RAZONES DE DESCARTE:

- No cumple los prerrequisitos.

Horario #14

DESCARTADO

Créditos Totales: 13

MATERIAS:

- Programación
- Matemáticas Discretas
- Matemáticas

RAZONES DE DESCARTE:

- Supera el máximo de créditos.